ZentroMart Voice Support Agent

VoiceOps 2026 capstone · LLM & Generative AI course, C&A Club IIT Guwahati x Gnani.ai
Aarna Rathod · BTech CSE, Manipal University Jaipur ·
github.com/aarnarathod114-lab/zentromart-agent

A support agent for ZentroMart that answers in English, Hindi and Hinglish from the company documents, cites the section IDs it used, looks up orders only after identity verification, hands over to a human when policy requires it, and works by voice through Gnani STT and TTS. One scorer (`score.py`) was used for every step, so all numbers are comparable.

Step	What I built	Result on the practice data
1 Prompt only	System prompt, no documents	30.1% of required facts, 76.9% decisions
2 RAG	Hybrid multilingual retrieval, business-day calendar	86.5% facts, 86.5% decisions, 86.9% retrieval recall
3 Conversation	Memory, identity-checked tools, handover tickets	92.7% decisions, 98.1% expected tool calls, 0 forbidden claims
4 Voice	Gnani STT/TTS, batch run command, live streaming	Transcript 2.0 s after the caller stops; all 5 sample calls handled

Architecture

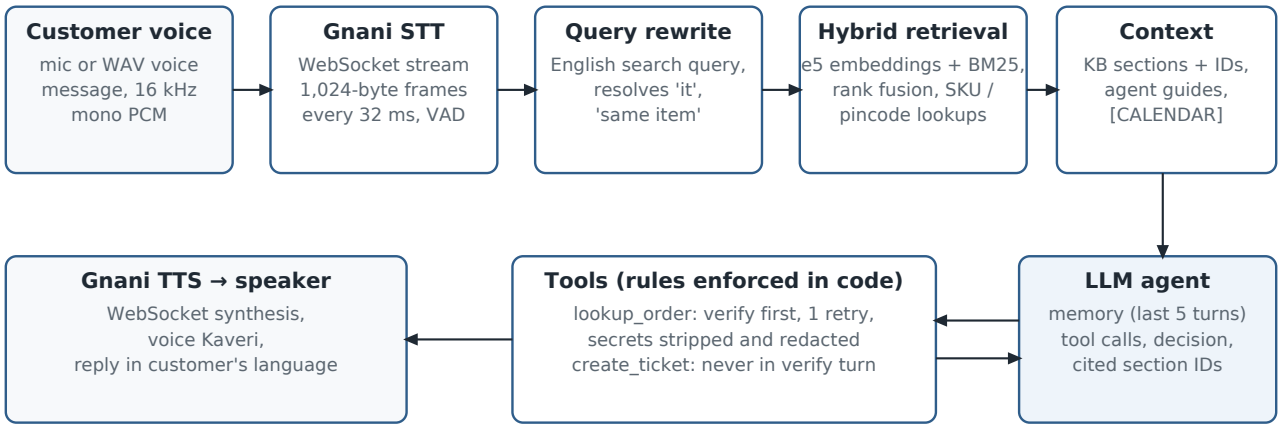


Figure 1. One customer turn. The same agent serves text evaluation and voice.

Steps 1 and 2: from guessing to grounded answers

Step 1. gpt-oss-120b (Groq) with a ZentroMart system prompt and no documents. It chose a sensible decision 76.9% of the time but produced only 30.1% of required facts: return windows, refund days and fees are company-specific, so it guessed. 11 of its 12 decision errors were "clarify" where an answer was expected.

Step 2. Each Markdown section becomes a chunk that keeps its [ID], doc_type, status and effective_from; each product row is a chunk too (154 chunks). Retrieval combines multilingual-e5-small embeddings (paraphrase, Hindi) with BM25 (exact SKUs and terms) by reciprocal rank fusion, plus a mild precedence prior (policy > agent guide > help > FAQ > marketing). A small model rewrites each question into an English search query, exact SKU/pincode rows are added, and business-day results are computed in Python as a [CALENDAR] block.

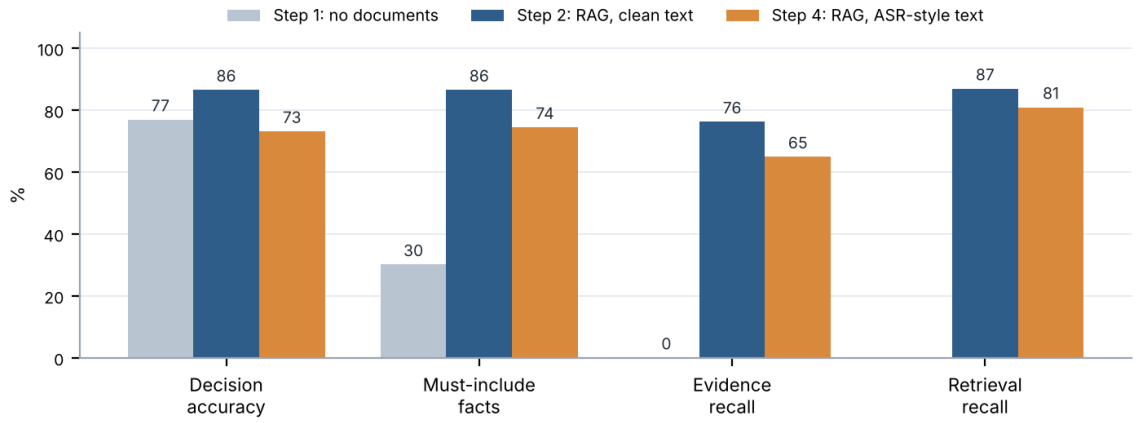


Figure 2. Same 52 questions and model (gpt-oss-120b). Step 1 retrieves nothing, so its recall is 0.

Metric (52 questions)	Step 1	Step 2	Step 2 on ASR-style text
Decision accuracy	76.9%	**86.5%**	73.1%
Must-include facts	30.1%	**86.5%**	74.4%
All required facts present	25.0%	**84.6%**	–
Evidence recall (cited IDs)	0%	**76.2%**	64.9%
Retrieval recall (gold IDs found)	–	**86.9%**	80.9%
Forbidden claims	0	0	0

By reply language, Step 2 reached 85% of facts in English, 100% in Hindi and 82% in Hinglish. **What I learned:** most of the gain came from retrieval itself, but each remaining error had a specific cause that small targeted tests exposed:

What went wrong	Why	Fix
Hindi refund question missed the refund policy	English policies, Hindi query	Also search with an English rewrite
SKU question lost the warranty policy	Catalog rows filled every slot	Separate slots for documents and catalog
Both models said 26 Oct, gold is 27 Oct	Counted the start day as day 1	Python calendar + explicit counting rule
Invented delivery-slot rules	General knowledge filled a gap	"Not described in context → escalate" rule

Step 3: holding a conversation

The agent keeps the last five exchanges plus a conversation state built from tool results, rewrites each message using recent turns (so "how long will that take?" resolves), always includes the agent guides and date rules, and calls `lookup_order` and `create_ticket`. Order dates feed the calendar, giving exact deadlines (sneakers delivered 12 Oct: return by 27 October 2026). Safety rules are **enforced in code**, not only prompted: the pincode and customer ID are stripped from the order before the model sees it and verification digits are redacted from replies and tickets; at most one retry; the decision label follows what the tools actually did; and no ticket can be created in the turn verification succeeds (the model kept escalating before the customer had asked anything).

23 conversations, 96 turns (Gemini Flash-Lite)	Result
Decision accuracy / must-include facts	**92.7%** / **88.0%**
Expected tool calls made (54 turns)	**98.1%**
Forbidden claims ("approved", leaked secrets)	**0**
Evidence recall / turns fully correct	70.5% / 82.3%

Results were similar across English, Hindi and Hinglish (81–83% fully correct turns).

Step 4: giving it a voice

`voice.py` uses the official gnani-vachana SDK: STT over WebSocket (16 kHz PCM, 1,024-byte frames every 32 ms) or REST for files, and TTS over WebSocket (voice Kaveri). **The run command** `python run.py --input audio_folder --output outputs` treats each file as a call and saves the transcript, decision, reply text and reply audio, cited section IDs and the full text of every retrieved chunk, plus `results.csv`. Unclear audio gets a spoken request to repeat. On five recorded sample calls it answered the electronics return window (7 days, with the 10-day exception), UPI refund time and the ₹25 COD fee in Hinglish, said "I don't have that information" for midnight delivery, and asked the caller to repeat the silent clip.

Voice stage	First version	Final version
Transcript after the caller stops	15.6–16.9 s (record, then stream)	**2.0 s** (mic streamed while speaking)
Agent (Groq gpt-oss-20b)	2.7–9.6 s	~3 s; up to 40 s when the free-tier per-minute limit hit
TTS synthesis	2.0–3.6 s	2.3–2.6 s

In a live call Gnani heard order "8978 2837" with the first 8 as a Hindi letter, giving a wrong ID. Verification failed, the agent allowed exactly one retry, then shared nothing and offered a human. The ASR-style practice text (e.g. "ele-zero zero one to" for ELE-0012) cost about 12 points, mainly on spoken identifiers, but produced no invented policies.

What went wrong and what I would fix

- **Spoken numbers** are the main voice weakness: normalise spoken digits back into SKUs and order IDs, and read an order number back to the caller before looking it up.
- **After two failed verifications** the agent offered a human but did not create the ticket; that handover should be created in code.
- **Over-clarifying and early escalation**: some answerable questions got "clarify", and a few repeat-contact cases escalated one turn early.
- **Free-tier limits** shaped the setup: Groq allows 200K tokens/day per model, so Step 3 (~600K tokens) ran on Gemini Flash-Lite; Gemini Flash took ~250 s per request and was unusable. Step 3 is therefore not a like-for-like model comparison with Step 2, and latencies include throttling.

Bonus work

- Real-time STT: microphone audio streamed to Gnani while the customer speaks (16 s → 2 s).
- ASR-robustness study on all 52 questions, comparing clean and speech-style text.
- Safety enforced in code (secret stripping and redaction, retry limit, ticket timing), resumable evaluations, and a Python business-day calendar.